Chapter 10. Leader Election

Synchronization can be quite costly: if each algorithm step involves contacting each other participant, we can end up with a significant communication overhead. This is particularly true in large and geographically distributed networks. To reduce synchronization overhead and the number of message round-trips required to reach a decision, some algorithms rely on the existence of the *leader* (sometimes called *coordinator*) process, responsible for executing or coordinating steps of a distributed algorithm.

Generally, processes in distributed systems are uniform, and any process can take over the leadership role. Processes assume leadership for long periods of time, but this is not a permanent role. Usually, the process remains a leader until it crashes. After the crash, any other process can start a new election round, assume leadership, if it gets elected, and continue the failed leader's work.

The *liveness* of the election algorithm guarantees that *most of the time* there will be a leader, and the election will eventually complete (i.e., the system should not be in the election state indefinitely).

Ideally, we'd like to assume *safety*, too, and guarantee there may be *at most one* leader at a time, and completely eliminate the possibility of a *split brain* situation (when two leaders serving the same purpose are elected but unaware of each other). However, in practice, many leader election algorithms violate this agreement.

Leader processes can be used, for example, to achieve a total order of messages in a broadcast. The leader collects and holds the global state, receives messages, and disseminates them among the processes. It can also be used to coordinate system reorganization after the failure, during initialization, or when important state changes happen.

Election is triggered when the system initializes, and the leader is elected for the first time, or when the previous leader crashes or fails to communicate. Election has to be deterministic: exactly one leader has to emerge from the process. This decision needs to be effective for all

participants.

Even though leader election and distributed locking (i.e., exclusive ownership over a shared resource) might look alike from a theoretical perspective, they are slightly different. If one process holds a lock for executing a critical section, it is unimportant for other processes to know who exactly is holding a lock right now, as long as the liveness property is satisfied (i.e., the lock will be eventually released, allowing others to acquire it). In contrast, the elected process has some special properties and has to be known to all other participants, so the newly elected leader has to notify its peers about its role.

If a distributed locking algorithm has any sort of preference toward some process or group of processes, it will eventually starve nonpreferred processes from the shared resource, which contradicts the liveness property. In contrast, the leader can remain in its role until it stops or crashes, and long-lived leaders are preferred.

Having a stable leader in the system helps to avoid state synchronization between remote participants, reduce the number of exchanged messages, and drive execution from a single process instead of requiring peer-to-peer coordination. One of the potential problems in systems with a notion of leadership is that the leader can become a bottleneck. To overcome that, many systems partition data in non-intersecting independent replica sets (see “Database Partitioning”). Instead of having a single system-wide leader, each replica set has its own leader. One of the systems that uses this approach is Spanner (see “Distributed Transactions with Spanner”).

Because every leader process will eventually fail, failure has to be detected, reported, and reacted upon: a system has to elect another leader to replace the failed one.

Some algorithms, such as ZAB (see “Zookeeper Atomic Broadcast (ZAB)”), Multi-Paxos (see “Multi-Paxos”), or Raft (see “Raft”), use temporary leaders to reduce the number of messages required to reach an agreement between the participants. However, these algorithms use their own algorithm-specific means for leader election, failure detection, and resolving conflicts between the competing leader processes.

Bully Algorithm

One of the leader election algorithms, known as the *bully algorithm*, uses process ranks to identify the new leader. Each process gets a unique rank assigned to it. During the election, the process with the highest rank becomes a leader [MOLINA82].

This algorithm is known for its simplicity. The algorithm is named *bully* because the highest-ranked node "bullies" other nodes into accepting it. It is also known as *monarchial* leader election: the highest-ranked sibling becomes a monarch after the previous one ceases to exist.

Election starts if one of the processes notices that there's no leader in the system (it was never initialized) or the previous leader has stopped responding to requests, and proceeds in three steps:[1]

1. The process sends election messages to processes with higher identifiers.

2. The process waits, allowing higher-ranked processes to respond. If no higher-ranked process responds, it proceeds with step 3. Otherwise, the process notifies the highest-ranked process it has heard from, and allows it to proceed with step 3.

3. The process assumes that there are no active processes with a higher rank, and notifies all lower-ranked processes about the new leader.

Figure 10-1 illustrates the bully leader election algorithm:

- a) Process `3` notices that the previous leader `6` has crashed and starts a new election by sending `Election` messages to processes with higher identifiers.

- b) `4` and `5` respond with `Alive`, as they have a higher rank than `3`.

- c) `3` notifies the highest-ranked process `5` that has responded during this round.

- d) `5` is elected as a new leader. It broadcasts `Elected` messages, notifying lower-ranked processes about the election results.

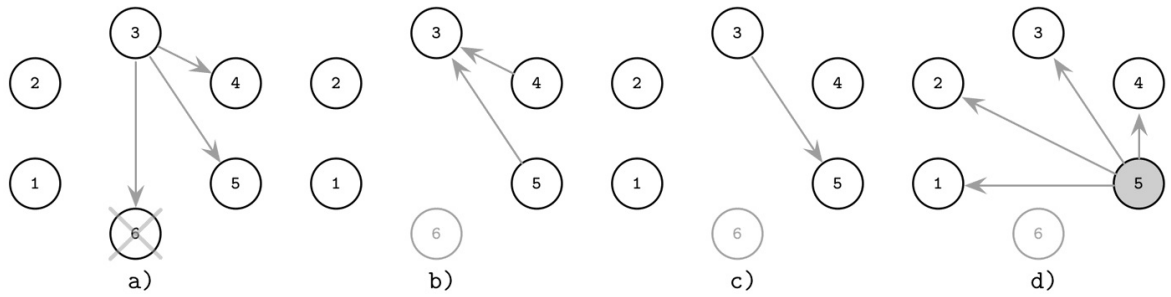

Figure 10-1. Bully algorithm: previous leader (6) fails and process 3 starts the new election

One of the apparent problems with this algorithm is that it violates the safety guarantee (that at most one leader can be elected at a time) in the presence of network partitions. It is quite easy to end up in the situation where nodes get split into two or more independently functioning subsets, and each subset elects its leader. This situation is called *split brain*.

Another problem with this algorithm is a strong preference toward high-ranked nodes, which becomes an issue if they are unstable and can lead to a permanent state of reelection. An unstable high-ranked node proposes itself as a leader, fails shortly thereafter, wins reelection, fails again, and the whole process repeats. This problem can be solved by distributing host quality metrics and taking them into consideration during the election.

Next-In-Line Failover

There are many versions of the bully algorithm that improve its various properties. For example, we can use multiple next-in-line alternative processes as a failover to shorten reelections [GHOLIPOUR09].

Each elected leader provides a list of failover nodes. When one of the processes detects a leader failure, it starts a new election round by sending a message to the highest-ranked *alternative* from the list provided by the failed leader. If one of the proposed alternatives is up, it becomes a new leader without having to go through the complete election round.

If the process that has detected the leader failure is itself the highest ranked process from the list, it can notify the processes about the new leader right away.

Figure 10-2 shows the process with this optimization in place:

- a) `6`, a leader with designated alternatives `{5,4}`, crashes. `3` notices this failure and contacts `5`, the alternative from the list with the highest rank.
- b) `5` responds to `3` that it's alive to prevent it from contacting other nodes from the alternatives list.
- c) `5` notifies other nodes that it's a new leader.

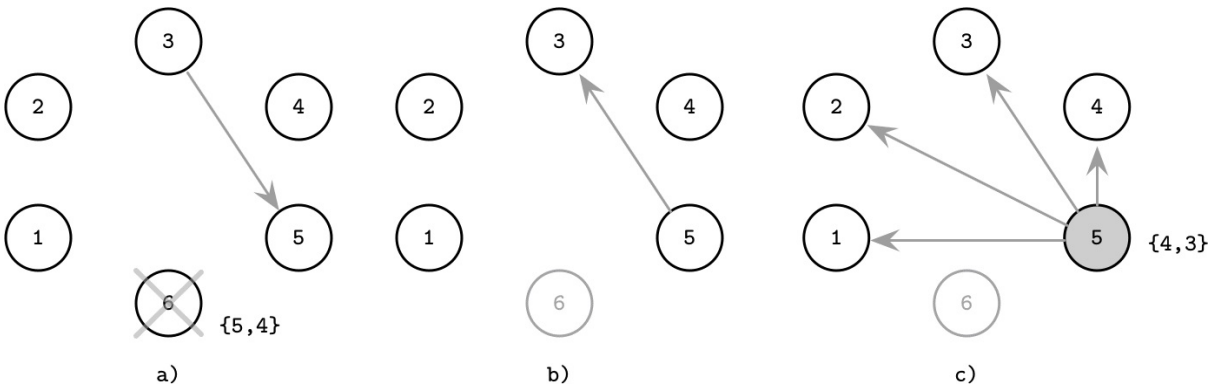

Figure 10-2. Bully algorithm with failover: previous leader (6) fails and process 3 starts the new election by contacting the highest-ranked alternative

As a result, we require fewer steps during the election if the next-in-line

process is alive.

Candidate/Ordinary Optimization

Another algorithm attempts to lower requirements on the number of messages by splitting the nodes into two subsets, *candidate* and *ordinary*, where only one of the candidate nodes can eventually become a leader [MURSHED12].

The ordinary process initiates election by contacting candidate nodes, collecting responses from them, picking the highest-ranked alive candidate as a new leader, and then notifying the rest of the nodes about the election results.

To solve the problem with multiple simultaneous elections, the algorithm proposes to use a tiebreaker variable δ, a process-specific delay, varying significantly between the nodes, that allows one of the nodes to initiate the election before the other ones. The tiebreaker time is generally greater than the message round-trip time. Nodes with higher priorities have a lower δ, and vice versa.

Figure 10-3 shows the steps of the election process:

- a) Process 4 from the ordinary set notices the failure of leader process 6. It starts a new election round by contacting all remaining processes from the candidate set.
- b) Candidate processes respond to notify 4 that they're still alive.
- c) 4 notifies all processes about the new leader: 2.

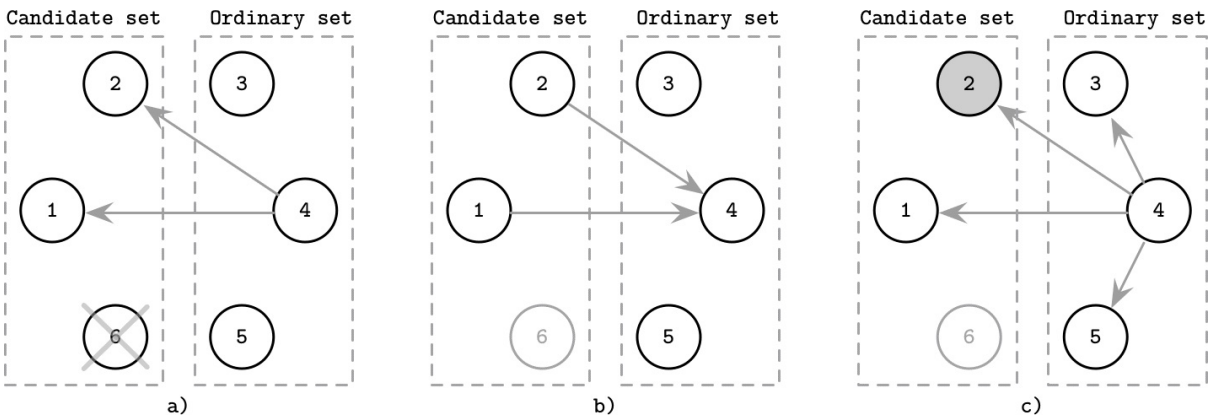

Figure 10-3. Candidate/ordinary modification of the bully algorithm: previous leader (6) fails and process 4 starts the new election

Invitation Algorithm

An *invitation algorithm* allows processes to “invite” other processes to join their groups instead of trying to outrank them. This algorithm allows multiple leaders *by definition*, since each group has its own leader.

Each process starts as a leader of a new group, where the only member is the process itself. Group leaders contact peers that do not belong to their groups, inviting them to join. If the peer process is a leader itself, two groups are merged. Otherwise, the contacted process responds with a group leader ID, allowing two group leaders to establish contact and merge groups in fewer steps.

Figure 10-4 shows the execution steps of the invitation algorithm:

- a) Four processes start as leaders of groups containing one member each. `1` invites `2` to join its group, and `3` invites `4` to join its group.
- b) `2` joins a group with process `1`, and `4` joins a group with process `3`. `1`, the leader of the first group, contacts `3`, the leader of the other group. Remaining group members (`4`, in this case) are notified about the new group leader.
- c) Two groups are merged and `1` becomes a leader of an extended group.

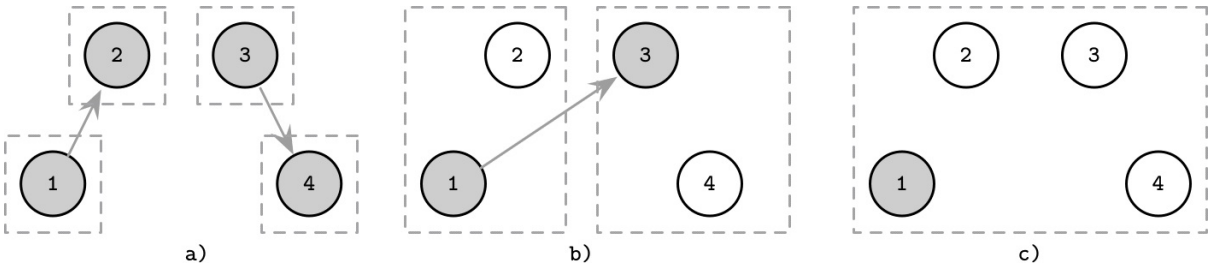

Figure 10-4. Invitation algorithm

Since groups are merged, it doesn’t matter whether the process that suggested the group merge becomes a new leader or the other one does. To keep the number of messages required to merge groups to a minimum, a leader of a larger group can become a leader for a new group. This way only the processes from the smaller group have to be notified about the

change of leader.

Similar to the other discussed algorithms, this algorithm allows processes to settle in multiple groups and have multiple leaders. The invitation algorithm allows creating process groups and merging them without having to trigger a new election from scratch, reducing the number of messages required to finish the election.

Ring Algorithm

In the ring algorithm [CHANG79], all nodes in the system form a ring and are aware of the ring topology (i.e., their predecessors and successors in the ring). When the process detects the leader failure, it starts the new election. The election message is forwarded across the ring: each process contacts its successor (the next node closest to it in the ring). If this node is unavailable, the process skips the unreachable node and attempts to contact the nodes after it in the ring, until eventually one of them responds.

Nodes contact their siblings, following around the ring and collecting the live node set, adding themselves to the set before passing it over to the next node, similar to the failure-detection algorithm described in "Timeout-Free Failure Detector", where nodes append their identifiers to the path before passing it to the next node.

The algorithm proceeds by fully traversing the ring. When the message comes back to the node that started the election, the highest-ranked node from the live set is chosen as a leader. In Figure 10-5, you can see an example of such a traversal:

- a) Previous leader `6` has failed and each process has a view of the ring from its perspective.
- b) `3` initiates an election round by starting traversal. On each step, there's a set of nodes traversed on the path so far. `5` can't reach `6`, so it skips it and goes straight to `1`.
- c) Since `5` was the node with the highest rank, `3` initiates another round of messages, distributing the information about the new leader.

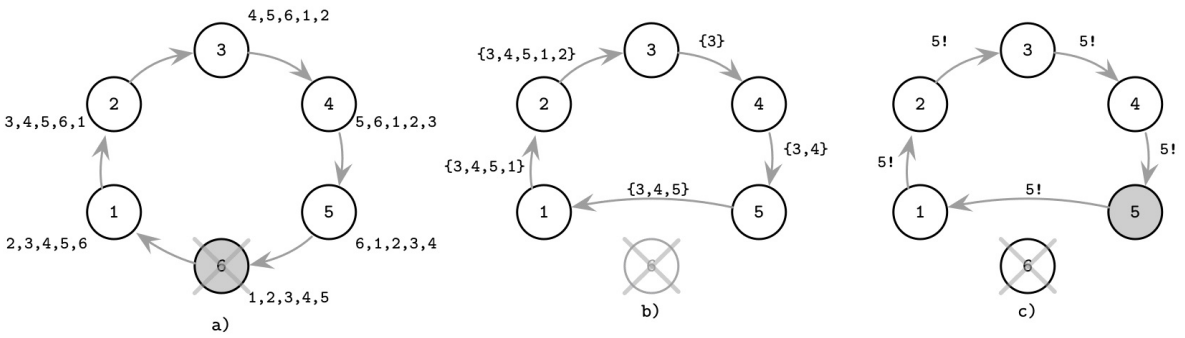

Figure 10-5. Ring algorithm: previous leader (6) fails and 3 starts the election process

Variants of this algorithm include collecting a single highest-ranked identifier instead of a set of active nodes to save space: since the `max` function is commutative, it is enough to know a current maximum. When the algorithm comes back to the node that has started the election, the last known highest identifier is circulated across the ring once again.

Since the ring can be partitioned in two or more parts, with each part potentially electing its own leader, this approach doesn't hold a safety property, either.

As you can see, for a system with a leader to function correctly, we need to know the status of the current leader (whether it is alive or not), since to keep processes organized and for execution to continue, the leader has to be alive and reachable to perform its duties. To detect leader crashes, we can use failure-detection algorithms (see Chapter 9).

Summary

Leader election is an important subject in distributed systems, since using a designated leader helps to reduce coordination overhead and improve the algorithm's performance. Election rounds might be costly but, since they're infrequent, they do not have a negative impact on the overall system performance. A single leader can become a bottleneck, but most of the time this is solved by partitioning data and using per-partition leaders or using different leaders for different actions.

Unfortunately, all the algorithms we've discussed in this chapter are prone to the split brain problem: we can end up with two leaders in independent subnets that are not aware of each other's existence. To avoid split brain, we have to obtain a cluster-wide majority of votes.

Many consensus algorithms, including Multi-Paxos and Raft, rely on a leader for coordination. But isn't leader election the same as consensus? To elect a leader, we need to reach a consensus about its identity. If we can reach consensus about the leader identity, we can use the same means to reach consensus on anything else [ABRAHAM13].

The identity of a leader may change without processes knowing about it, so the question is whether the process-local knowledge about the leader is still valid. To achieve that, we need to combine leader election with failure detection. For example, the *stable leader election* algorithm uses rounds with a unique stable leader and timeout-based failure detection to guarantee that the leader can retain its position for as long as it doesn't crash and is accessible [AGUILERA01].

Algorithms that rely on leader election often *allow* the existence of multiple leaders and attempt to resolve conflicts between the leaders as quickly as possible. For example, this is true for Multi-Paxos (see "Multi-Paxos"), where only one of the two conflicting leaders (proposers) can proceed, and these conflicts are resolved by collecting a second quorum, guaranteeing that the values from two different proposers won't be accepted.

In Raft (see "Raft"), a leader can discover that its term is out-of-date, which implies the presence of a different leader in the system, and update

its term to the more recent one.

In both cases, having a leader is a way to ensure *liveness* (if the current leader has failed, we need a new one), and processes should not take indefinitely long to understand whether or not it has really failed. Lack of *safety* and allowing multiple leaders is a performance optimization: algorithms can proceed with a replication phase, and *safety* is guaranteed by detecting and resolving the conflicts.

We discuss consensus and leader election in the context of consensus in more detail in Chapter 14.

FURTHER READING

If you'd like to learn more about the concepts mentioned in this chapter, you can refer to the following sources:

Leader election algorithms

Lynch, Nancy and Boaz Patt-Shamir. 1993. "Distributed algorithms." *Lecture notes for 6.852*. Cambridge, MA: MIT.
Attiya, Hagit and Jennifer Welch. 2004. *Distributed Computing: Fundamentals, Simulations and Advanced Topics*. USA: John Wiley & Sons.
Tanenbaum, Andrew S. and Maarten van Steen. 2006. *Distributed Systems: Principles and Paradigms* (2nd Ed.). Upper Saddle River, NJ: Prentice-Hall.

1 These steps describe the *modified* bully election algorithm [KORDAFSHARI05] as it's more compact and clear.